Make type-erased allocator use in `promise` and `packaged_task` consistent

Document #:	P3503R1
Date:	2025-03-16
Project:	Programming Language C++
Audience:	Library

Reply-to:	Nicolas Morales <nmmoral@sandia.gov> Jonathan Wakely <cxx@kayari.org>

Contents

§ 1 Revision History

§ 1.1 P3503R1: 2025-03 Post-Hagenberg Mailing

— Forwarded by LEWG to LWG after a reflector review with 15 votes, no objections

— Fix wording after mailing list review

§ 1.2 P3503R0: 2024-12 Post-Wrocław Mailing

— Initial paper revision

§ 2 Motivation and Background

[LWG2095] points out inconsistencies in the API of `std::promise`; namely that `uses_allocator<promise<R>, A>::value` is specified to be true in 32.10.6 [futures.promise], yet is missing the appropriate constructor taking an rvalue of promise.

This example shows the problem [1]:

```
using prom = promise<void>;

tuple<prom> t1{ allocator_arg, a };
tuple<prom> t2{ allocator_arg, a, prom{} }; // ill-formed
```

Meanwhile, [LWG2921] and [LWG2976] removed the allocator constructors from `packaged_task`; a followup issue [LWG3003] originally suggested removing them from `promise`. Part of the motivation for this was that although the original reasoning for removing allocator constructors from `packaged_task` was incorrect (to keep the design parallels between `std::packaged_task` and `std::function`), LWG decided to keep this resolution as it was a response to an NB comment.

In Varna, LEWG wanted a resolution to this issue that keeps the constructor in `std::promise` as it is actually useful (and removing it would break existing code regardless). Furthermore, LEWG indicated that to resolve [LWG2095] that the the `uses_allocator` specialization should be removed from `std::promise`, since the extra parameter to the moving constructor would have been ignored anyway. Finally, the constructors incorrectly removed from `std::packaged_task` would be restored, though `uses_allocator` would not be re-added for `std::packaged_task`.

Re-adding the constructor from `std::packaged_task` re-raises another issue ([LWG2245]) that was closed with the resolution of [LWG2921]. Basically, `packaged_task::reset()` did not allow a user to supply an allocator even though an allocator was used. In the issue discussion it was decided that the best resolution would be for using the allocator that was provided to `std::packaged_task` in construction, rather than adding an additional overload to reset that takes an allocator.

Also in Varna, it was resolved to write a paper to introduce these changes. This paper (I suppose) never materialized, and [LWG2095] and [LWG3003] were brought before LEWG in Wroclaw in 2024. This paper is intended to bring together the resolution and issues into a single paper for voting on by LEWG.

§ 3 Design

The design of this paper is intended to resolve the LWG issues as follows:

1. Resolve [LWG2095] by removing the `std::uses_allocator` specialization for `std::promise`, but do not remove the existing constructor.
2. Resolve [LWG3003] by re-introducing the previously removed constructors

3. Avoid reintroducing a similar bug to [LWG2095] by not re-adding a `std::uses_allocator` specialization to `std::packaged_task`

4. Ensure that `std::packaged_task::reset()` uses the allocator it was constructed with to create the new shared state.

This paper adopts the wording changes in the final suggested revision to [LWG3003], rebased on the latest draft at the time of this writing ([N4993]).

§ 3.1 Alternatives

There were two variant designs suggested in these issues, but were rejected in favor of other designs and I wouldn't favor adopting them:

1. Add a new constructor to `std::promise` (and `std::packaged_task`) that takes an allocator and rvalue reference. This was not the alternative that LEWG decided on in Varna. This is the original resolution to [LWG2095] that was abandoned.

2. Add a new overload for `std::packaged_task::reset()` that takes an allocator. I'm not sure how useful this would actually be to use a different allocator for the new shared object.

§ 4 Wording

Modify 32.10.6 [futures.promise] as indicated:

```
~~template<class R, class Alloc>~~
  ~~struct uses_allocator<promise<R>, Alloc>;~~
```

[…]

```
~~template<class R, class Alloc>~~
  ~~struct uses_allocator<promise<R>, Alloc>~~
    ~~: true_type { };~~
```

~~*Preconditions:* Alloc meets the *Cpp17Allocator* requirements (16.4.4.6.1 [allocator.requirements.general]).~~

Modify 32.10.10.1 [futures.task.general] as indicated:

```
template<class R, class... ArgTypes>
class packaged_task<R(ArgTypes...)> {
public:
  // construction and destruction
  packaged_task() noexcept;
  template<class F>
    explicit packaged_task(F&& f);

  template<class F, class Allocator>
    explicit packaged_task(allocator_arg_t, const Allocator& a, F&& f);
  ~packaged_task();
```

Modify 32.10.10.2 [futures.task.members] as indicated:

```
template<class F>
  explicit packaged_task(F&& f);
```

2 *Effects:* Equivalent to `packaged_task(allocator_arg, allocator<int>(), std::forward<F>(f))`.

> [*Drafting note:* Uses of `allocator<int>` and `allocator<unspecified>` are not observable so this constructor can be implemented without delegating to the other constructor and without using `allocator`.]

```
template<class F, class Allocator>
  packaged_task(allocator_arg_t, const Allocator& a, F&& f);
```

2 *Constraints:* `remove_cvref_t<F>` is not the same type as `packaged_task<R(ArgTypes...)>`.

3 *Mandates:* `is_invocable_r_v<R, F&, ArgTypes...>` is `true`.

4 *Preconditions:* Invoking a copy of `f` behaves the same as invoking `f`. `Allocator` meets the *Cpp17Allocator* requirements (16.4.4.6.1 16.4.4.6.1 [allocator.requirements.general]).

5 *Effects:* Let `A2` be `allocator_traits<Allocator>::rebind_alloc<unspecified>` and let `a2` be an lvalue of type `A2` initialized with `A2(a)`. Constructs a new `packaged_task` object with a shared state and initializes the object's stored task with `std::forward<F>(f)`. Uses `a2` to allocate storage for the shared state and stores a copy of `a2` in the shared state.

6 *Throws:* ~~Any exceptions thrown by the copy or move constructor of `f`, or `bad_alloc` if memory for the internal data structures cannot be allocated.~~ Any exceptions thrown by the initialization of the stored task. If storage for the shared state cannot be allocated, any exception thrown by `A2::allocate`.

…

```
void reset();
```

26 *Effects:* ~~As if~~ Equivalent to:

```
if (!valid())
  throw future_error(future_errc::no_state);
*this = packaged_task(allocator_arg, a,std::move(f))
```

where `f` is the task stored in `*this` and `a` is the allocator stored in the shared state.

[*Note 2:* This constructs a new shared state for `*this`. The old state is abandoned (32.10.5 [futures.state]). — *end note*]

27 *Throws:*

(27.1) ~~— `bad_alloc` if memory for the new shared state cannot be allocated.~~

(27.2) — Any exception thrown by the `packaged_task` constructor~~move constructor of the task stored in the shared state~~.

(27.3) — `future_error` with an error condition of `no_state` if `*this` has no shared state.

§ 5 Acknowledgments

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525. This paper describes objective technical results and analysis. Any subjective views or opinions that might be expressed in the paper do not necessarily represent the views of the U.S. Department of Energy or the United States Government.

§ 6 References

[LWG2095] Jonathan Wakely. promise and packaged_task missing constructors needed for uses-allocator construction.
https://wg21.link/lwg2095

[LWG2245] Jonathan Wakely. packaged_task::reset() memory allocation.
https://wg21.link/lwg2245

[LWG2921] United States. packaged_task and type-erased allocators.
https://wg21.link/lwg2921

[LWG2976] Tim Song. Dangling uses_allocator specialization for packaged_task.
https://wg21.link/lwg2976

[LWG3003] Billy O'Neal III. <future> still has type-erased allocators in promise.
https://wg21.link/lwg3003

[N4993] Thomas Köppe. 2024-10-16. Working Draft, Programming Languages — C++.
https://wg21.link/n4993

---

1. This example does compile with `libstdc++` because it preemptively incorporates a fix to the problem.↩